```
puts Child.new.execute
```

produces:

```
logging
childing
calling
parenting
```

When the execute method is called, Ruby looks first at the prepended module, Log, and executes there. That method calls super, which continues the lookup chain upward, to the actual Child class. The super in that method moves up the lookup chain to the included module Caller, and the super in that method moves up the chain to Parent.

Inheritance, Mixins, and Design

Inheritance and mixed-in modules both allow you to write code in one place and use that code in multiple classes. So, when do you use each?

As with most questions of design, the answer is, well…it depends. However, over the years developers have come up with some general guidelines to help us decide.

First let's look at subclassing. Classes in Ruby are related to the idea of types. It would be natural to say that "cat" is a string and [1, 2] is an array. And that's another way of saying that the class of "cat" is String and the class of [1, 2] is Array. When we create our own classes, you can think of it as adding new types to the language. And when we subclass either a built-in class or our own class, we're creating a *subtype*.

Now, a lot of research has been done on type theories. One of the more useful concepts is the *Liskov Substitution Principle*. The Liskov Substitution Principle states that you should be able to substitute an object of a child class wherever you use an object of the parent class—the child should honor the parent's contract. There's another way of looking at this relationship: we should be able to say that the child object *is a* kind of the parent. We're used to saying this in English: a car *is a* vehicle, a cat *is an* animal, and so on. This means that a cat should, at the very least, be capable of doing everything we say that a generic animal can do.

So, when you're looking for subclassing relationships while designing your application, be on the lookout for these *is-a* relationships.

But…here's the bad news. In the real world, there really aren't that many true *is a* relationships. Instead, it's far more common to have *has a* or *uses a* relationships between things. The real world is built using composition, not strict hierarchies.

In the past, we've tended to gloss over that fact when programming. When inheritance is the only scheme available for sharing code, it's easy to say things like "My Person class is a subclass of my DatabaseWrapper class." (Indeed, the Ruby on Rails framework makes just this design choice.) But a person object *is not* a kind of database wrapper object. A person object *uses* a database wrapper to provide persistence services.

Is this just a theoretical issue? No! Inheritance represents an incredibly tight coupling of two components. Change a parent class, and you risk breaking the child class. But, even worse, if code that uses objects of the child class relies on those objects also having methods defined in the parent, then all that code will break, too. The parent class's implementation leaks through the child classes and out into the rest of the code. With a decent-sized program, this becomes a serious inhibitor to change.